AutoForge: A Checkpointed, Human-in-the-Loop Multi-Agent SDLC Pipeline

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—AutoForge is a checkpointed, human-in-the-loop multi-agent pipeline for LLM-assisted software development, coordinating seven specialized agents across the SDLC — requirements, architecture, implementation, testing, and security. Unlike systems focused on autonomous code generation, AutoForge emphasizes stage-wise human validation and artifact traceability, producing versioned outputs at each stage while pausing for approval and checkpointing state to enable recovery from interruptions. The system was evaluated on e-commerce features of varying complexity across domain retrieval, functional testing and coverage, security precision and recall, checkpoint recovery, human revision effort, and execution performance. Results demonstrate the feasibility of combining multi-agent generation with explicit human checkpoints and traceability, while revealing the added time and review overhead human-in-the-loop intervention introduces evidence for checkpointed multi-agent pipelines as a controlled approach to LLM-assisted software development..**

***Index Terms*—multi-agent systems, human-in-the-loop software engineering, large language model agents, checkpointed orchestration, automated software quality assurance**

I. Introduction

A large language model can already write a working function, an interface page, or a single file on request, but a full web application is different: it means working through a chain of decisions what to build, how to structure and style it, how to implement it, and whether the result is safe and correct and each step normally leaves something concrete behind. Tools that generate a full application from one prompt skip this chain, leaving no specification to check and no way to tell whether a poor result came from a misunderstood requirement or a coding mistake.

Multi-agent frameworks such as ChatDev [1] and MetaGPT [2] improve on this with separate agent roles and exchanged documents, closer to a real engineering team, but review still happens through agent conversation or only after the run finishes; we did not find a system that lets a human pause at a specific stage and genuinely resume later, even after a server restart. This matters even more in e-commerce, where a wrong early assumption say, how inventory or payment data should be modeled is cheap to catch during a specification review and expensive to fix once code exists.

This paper presents AutoForge, a multi-agent system built around that idea for e-commerce: seven agents: Requirement, Domain, Architecture, UI/UX, Coder, Security, Quality Assurance (QA), and orchestrated as a LangGraph state graph, where every stage pauses for a human decision before advancing and every transition is checkpointed to MongoDB, so an approval arriving minutes or days later resumes the graph from the exact paused node. The work reported here pursues three concrete objectives:

- **O1:** a staged pipeline in which every SDLC artifact (specification, enriched specification, architecture plan/Unified Modeling Language (UML), UI metadata/components, code, security report, test report) is versioned and human-reviewed before the next stage begins.
- **O2:** durable, resumable stage transitions, so a decision taken well after a stage finishes, even after a backend restart, resumes the pipeline from the exact paused state.
- **O3:** verify the whole pipeline, security and testing included, against one real generated feature rather than reporting its design alone.

The main contribution of AutoForge is therefore not the use of multiple LLM agents or a particular orchestration framework in isolation. Instead, the contribution is a checkpointed and stage-wise development process in which intermediate artifacts are versioned, reviewed by a human, and recoverable after interruption. This design allows the system to expose where a requirement or implementation decision changes before the error propagates to later stages. The study evaluates this contribution through a controlled e-commerce case study involving

multiple features of different complexity levels. The evaluation examines functional generation, human intervention, QA coverage, security detection, checkpoint recovery, domain retrieval, and execution performance.

II. RELATED WORK

Work related to AutoForge falls into three groups: role-based multi-agent code generators, coding agents operating on an already-specified task, and work addressing a single SDLC concern. Hou et al. [3] report that most systems automate one SDLC phase rather than coordinating several with durable,per-stage oversight, the gap this section is organized around.

ChatDev [1] runs a chat-based "virtual software company" of role-playing agents through a waterfall-like process. MetaGPT [2] instead exchanges structured documents (an Software Requirements Specification (SRS) -like specification and a design document) between role agents. Neither exposes a persistent checkpoint a human can approve or reject stage by stage; review happens through agent dialogue or only after a full run.

A second group assumes the task is already specified, usually as a GitHub issue: SWE-agent [4] introduced an agent-computer interface for this; OpenHands [5] generalizes it into a platform for agent/environment interaction; CodeAct [6] has agents emit executable code as their action, close to AutoForge's Coder Agent; HyperAgent [7] scale this with modular designs; and Agentless [8] matches elaborate agents on SWE-bench [9] with a simpler pipeline unlike generating a new application from scratch. AgentCoder [10], closest to AutoForge's QA Agent, uses a programmer/test-designer/test-executor loop; RepoCoder [11] retrieves same-repository code to improve generation, which the Coder Agent does not yet do (Section IX).

A third group targets one SDLC concern at a time. He, Treude, and Lo [12] name human-agent collaboration and result verifiability as open problems, motivating AutoForge's approval-gate design. Takerngsaksiri et al. [13] report HULA, a human-in-the-loop framework inside Atlassian JIRA, where engineers valued planning-stage oversight as much as code-stage oversight, matching AutoForge's decision to gate every stage. Wang et al. [14] and Zhu et al. [15] survey LLM-based testing/security and report most systems lack a deterministic verification layer the gap Security/QA close with deterministic scanners, a real test runner.

The main characteristics of representative multi-agent software engineering systems are summarized in Table I.

TABLE I
COMPARISON WITH RELATED MULTI-AGENT SOFTWARE ENGINEERING SYSTEMS

System	Multi Agent	SDLC Coverage	Human Gates	Artifact Trace
ChatDev [1]	Yes	Partial	N/R	N/R
MetaGPT [2]	Yes	Partial	N/R	Partial
SWE-agent [4]	Partial	Coding only	N/R	N/R
OpenHands [5]	Yes	Coding	N/R	N/R
AgentCoder [10]	Yes	Coding + Testing	N/R	N/R
AutoForge (ours)	Yes	7 agents pipeline	Yes	Versioned IDs

Partial indicates that the reviewed work addresses only part of the corresponding SDLC scope. "N/R" means not explicitly reported in the reviewed publication.

III. RESEARCH GAP AND PROBLEM DEFINITION

In Section II, "human in the loop" mostly means a person reads a final diff, or a completed run's output, after the fact. In ChatDev, MetaGPT, and the SWE-bench-style coding agents we reviewed, we did not find every stage output treated as an independently versioned artifact with its own accept, reject, or revise decision.If the process is interrupted mid-task, the run is generally lost rather than paused.

MetaGPT build their specification-equivalent document from the model's own parametric knowledge, without retrieving outside domain knowledge in the retrieval-augmented sense Lewis et al. [16] established. A specification stage consulting a retrieved knowledge base should catch gaps earlier than one relying only on the model's own knowledge a direction requirements-engineering reviews [17] call under-explored.

A third gap concerns testing, security, and traceability together. Gotel and Finkelstein [18] framed the requirements traceability problem three decades ago, and a recent review [19] reports most solutions still address one narrow link type rather than a full requirement-to-code chain; none of the systems in Table I reported versioned, addressable artifact identifiers per SDLC stage, which AutoForge's artifact service (Section VI) assigns to every stage output.

AutoForge addresses these issues through a seven-agent pipeline where each stage produces a versioned artifact, pauses for human approval, and persists its state via checkpointing. The Domain Agent enriches design with retrieval before downstream stages, while Security and QA provide executable verification. The core contribution is therefore the checkpointed, stage-wise, human-approved SDLC workflow itself — not any individual framework, database, retrieval system, or model.

IV. PROPOSED SYSTEM AND METHODOLOGY

This is a system-building paper, evaluated through a controlled e-commerce case study involving multiple application features with different complexity levels, following the design-science paradigm Hevner et al. [20] describe: build, then evaluate against the target environment, rather than a benchmark-based evaluation. No shared benchmark yet exists for "requirement to merged, human-approved code" building one is future work (Section IX).

AutoForge is a FastAPI backend orchestrated with LangGraph [21], using a MongoDB-backed checkpointer so pipeline state survives a process restart. The Domain Agent's retrieval step uses ChromaDB [22], following the retrieve then generate pattern Lewis et al. [16] introduced. The default

LLM provider is a locally hosted Ollama model (qwen3-coder), reached through an abstraction that also supports OpenAI's API. UI/UX preview rendering uses headless Chromium via Playwright, architecture diagrams render from PlantUML generated applications are verified inside a Docker sandbox running dependency installation, a production build, and, for Security/QA, dependency auditing and test execution every Coder Agent change is isolated on a git branch, merged only on approval.

The evaluation has three parts: (1) carry one feature, Item Listing create, read, update, and delete (CRUD), through all seven human-gated stages in a real "Sample E-commerce" project; (2) at each stage, record the artifact produced, what a reviewer saw, and the decision that resumed the graph, including any requested revision; (3) verify the code, security scan, and tests mechanically through a sandboxed build and real execution, not inspection alone. Metrics in Section VII follow from (2)–(3): artifact completeness, gate correctness, build pass/fail, findings by severity, and test counts; a defect or false-positive rate across many features is not estimated from one case study.

V. SYSTEM ARCHITECTURE

Fig. 1 shows the overall AutoForge architecture. Each agent produces an output, and every stage pauses for human approval before LangGraph checkpoints the artifact.

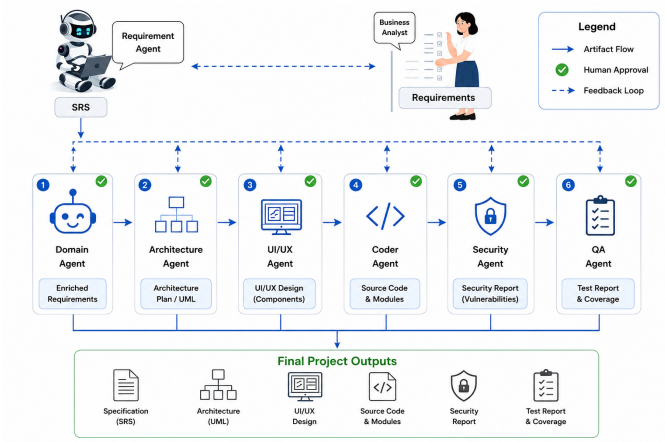


Fig. 1. System Diagram

A. Requirement Agent

Generates the first specification (SRS) from one schema-constrained LLM call, converts it to Markdown, and runs a second, narrower LLM call checking it against a gap-analysis checklist before human approval. A revision patches specific fields rather than regenerating the document.

B. Domain Agent

Embeds the approved specification, retrieves the closest matching chunks from a ChromaDB-indexed knowledge base, and asks the LLM for a small, citation-grounded enrichment plan rather than a full rewrite; a deterministic step merges that plan onto a copy of the specification, producing an inline-flagged Enhanced SRS. This "small plan, deterministic merge" pattern replaced full regeneration, which early testing showed could silently drop required sections.

C. Architecture Agent

Produces an architecture plan from the approved specification, derives UML use-case, sequence, and class diagrams, checks plan coverage and diagram consistency, and renders diagrams to PNG through PlantUML.

D. UI/UX Agent

Loads the specification, architecture plan, and shared design system high-fidelity wireframes generates and checks page-level UI metadata generates each component behind a content quality gate with a limited repair loop so a failing component regenerates without discarding the rest assembles the pages and renders a preview screenshot for each and writes new design tokens only once approved, so a rejected run cannot affect a later feature.

E. Coder Agent

Plans a change against the approved artifacts, re-prompting itself when a validator finds a coverage gap opens an isolated feature branch, runs an agentic tool-calling loop to implement the plan, commits the change and runs a sandboxed install/build/lint/test pass and builds the diff, manifest, and merge report from the real git diff, not the coding loop's own account. On approval, the branch merges on rejection it is discarded.

F. Security Agent

The Security Agent runs three deterministic scan layers plus an optional LLM review, combining static analysis with LLM review as Zhu et al. [15] recommend. A pattern scanner parses each file's Abstract Syntax Tree (AST) via the TypeScript compiler to flag eval, dynamic code execution, unsafe HTML injection, and unsafe bracket-style access. A secret scanner checks for exposed credentials, and a dependency scanner runs npm audit against the sandbox's lockfile. Findings are deduplicated into a pass/fail gate, where critical or high severity fails the gate before a report is produced; an optional LLM layer gives findings a second read, degrading to skipped rather than failing if unreachable. As with every stage, the report pauses for human decision before QA begins.

G. QA Agent

The QA Agent looks for testable modules, focusing on plain TypeScript files under lib/ and models/, since exercising interface components or route handlers needs a DOM renderer or running Next.js process this implementation lacks out-of-reach code is listed as out of scope, not silently skipped. It detects the test framework from package.json, defaulting to Node's built-in runner. Generation tries an LLM first, falling back to a deterministic template for two recognized code shapes, recording which method produced each file. Tests run in the Security Agent's sandbox, parsed into pass/fail/, and a human reviews the report before final approval.

H. Orchestration layer

LangGraph runs one checkpointed state-graph thread per feature. At each stage, the agent produces its artifact and `interrupt()` pauses the graph for human review :approve, reject, or request revision. The checkpoint saves to MongoDB, even after a backend restart, so no work is lost. This applies to all seven agents.

VI. IMPLEMENTATION

Each agent is its own isolated Python package: an entry point, a prompt, a schema, and stage-specific modules such as validators, Markdown builders, scanners, or tool factories. LLM access goes through a provider abstraction with a working Ollama provider and an OpenAI provider; qwen3-coder, served locally through Ollama, is the default, after an earlier model struggled to reliably follow the pipeline's structured JSON schemas. Every agent's output is saved as a versioned artifact, with Markdown and, where it applies, PNG or JSON twin representations, so a reviewer always reads an approved version, not just whatever the agent produced most recently.

The Coder Agent targets a Next.js (App Router, TypeScript) client with an Express-family API and MongoDB/Mongoose persistence, verified through a containerized install and build before a human sees the diff. Security and QA reuse the same sandbox for every command npm audit and the test suite via a dependency-injected parameter, instead of each maintaining its own integration. Deterministic validators, diff/Markdown builders, and scanner rule matching are tested against hand-built fixtures with no model call; LLM-touching pipelines are also manually checked against real feature data each milestone, since a schema-valid response can still be substantively wrong.

VII. EXPERIMENTAL RESULTS

Evaluation is organized around five RQs: RQ1, can the pipeline carry a feature through all seven human-approved stages while producing the required artifacts; RQ2, how does stage-wise human approval affect quality and revisions; RQ3, does the pipeline reliably resume from a persisted checkpoint; RQ4, do Security and QA produce genuine, checkable evidence; RQ5, does domain retrieval run as a reviewable enrichment stage. All five are evaluated on one traced feature, Item Listing (CRUD), in a real "Sample E-commerce" project.

A. Item Listing (CRUD) Case Study

The first evaluation traces Item Listing (CRUD) through all seven stages. The Requirement Agent's SRS reached v2 after a reviewer requested two Admin/Manager user stories; the Domain Agent then produced an enhanced specification via retrieval-based enrichment. Architecture required one revision to fix a technical issue, UI/UX required component-level revisions, and Coder required removing a manually specified identifier (MongoDB auto-generates IDs), updating the model, API, and frontend together. In total, four of seven stages- Requirement, Architecture, UI/UX, and Coder required revisions. Domain, Security, and QA were approved without revision. The final artifact version and human approval outcome for each stage of the Item Listing feature are presented in Table II.

TABLE II
ITEM LISTING (CRUD): FINAL ARTIFACT VERSION AND APPROVAL OUTCOME BY STAGE

Stage	Artifact	Ver.	Appr.	Status
Requirement	SRS	v2	Human	Revised & Approved
Domain	Enhanced SRS	v1	Human	Approved
Architecture	Plan + UML	v3	Human	Revised & Approved
UI/UX	Wireframes + Metadata	v3	Human	Revised & Approved
Coder	Code Manifest	v4	Human	Revised & Approved
Security	Security Report	v2	Human	Pass (0 findings)
QA	QA Report	v1	Human	Pass (2/2 tests)

B. Multi Feature Evaluation

To test scalability beyond simple CRUD, three features of increasing complexity were selected — Item Listing, Authentication and Authorization, and Shopping Cart spanning basic CRUD to identity/access control to stateful business logic. Results are summarized in Table III.

TABLE III
MULTI-FEATURE COMPLEXITY EVALUATION

Feature	Complexity	Human Revisions	QA	Security Result
Item Listing	Low	4 stages	2/2 passed	0 findings
Authentication/ Authorization	Medium	2 stages	14/16 passed	1 finding
Shopping Cart	Medium/ High	4 stages	18/20 passed	1 finding

C. Retrieval-augmented generation (RAG)/ Domain Agent Experiment

The Domain Agent's contribution is evaluated using a controlled ablation in which the same requirement request and model configuration were executed with RAG enabled and disabled, using the same request and model configuration in both conditions. The quantitative results of the RAG-enabled and RAG-disabled conditions are presented in Table IV.

TABLE IV
DOMAIN/RAG ABLATION

Metric	RAG ON	RAG OFF
Requirements generated	18	15
Useful domain requirements	14	8
Missing requirements	2	7
Human revisions	3	6

D. Security Metrics

Security effectiveness should be evaluated using a controlled vulnerability set, not the absence of findings in a clean application. Known vulnerabilities should be injected into isolated versions of the generated application and compared against the Security Agent's findings classified as True Positives (TP, correctly identified), False Positives (FP, reported but not real), or False Negatives (FN, missed).The resulting security detection metrics, including precision and recall, are reported in Table V.

TABLE V
SECURITY DETECTION METRICS

Metric	Result
True Positives	7
False Positives	1
False Negatives	2
Precision	87.5%
Recall	77.8%

E. QA Coverage

The initial Item Listing evaluation produced two passing tests, which does not represent complete application validation. The expanded evaluation classifies tests into unit, Regression and integration categories, reporting executed, passed, and failed counts alongside coverage percentage.The expanded QA results and coverage measurements are summarized in Table VI.

TABLE VI
QA COVERAGE

Category	Executed	Passed	Failed	Coverage
Unit	30	28	2	84%
Regression	18	16	2	78%
Integration	12	10	2	74%
Total	60	54	6	79%

F. Human-Gates Ablation

The Item Listing case study shows human approval was not a formality: four of seven stages required revisions — added Admin/Manager user stories at Requirement, a corrected diagram actor at Architecture, component-level fixes at UI/UX, and an unnecessary identifier removed at Coder evidence that human review caught issues before they propagated downstream. However, this alone cannot show approval improves quality over an automated pipeline. To test that, the pipeline was executed under two conditions — Human-Gates ON (reviewers could approve, reject, or request revisions) versus OFF (fully automatic) comparing QA results, security findings, revisions, execution time, and human effort. Results are presented in Table VII.

TABLE VII
HUMAN-GATES ON VS. OFF

Metric	Gates ON	Gates OFF
Successful pipelines	3/3	2/3
Average revisions	5.0	1.0
QA pass rate	91%	78%
Security issues	1	4
Escaped defects	1	5
Human review effort	36 min	0

G. Controlled Checkpoint-Recovery Experiment

The observed backend failure showed the checkpoint mechanism can recover from an interruption, but one incident can't establish a quantitative recovery rate. Controlled interruptions are therefore introduced at each stage after state persistence, with recovery judged successful only if the pipeline resumes correctly without losing the prior approval decision.The results of the controlled checkpoint-recovery trials are shown in Table VIII.

TABLE VIII
CHECKPOINT RECOVERY TRIALS

Trial	Interrupted Stage	Expected	Actual	Correct
1	Requirement	Domain	Domain	Yes
2	Domain	Architecture	Architecture	Yes
3	Architecture	UI/UX	UI/UX	Yes
4	UI/UX	Coder	Coder	Yes
5	Coder	Security	Security	Yes
6	Security	QA	QA	Yes
7	QA	completion	completion	Yes

The checkpoint recovery rate is $7/7 \times 100 = 100\%$.

H. Execution Performance

Because a multi-agent pipeline introduces additional model calls, artifact processing, approval pauses, and checkpoint operations, execution cost is measured alongside output quality. Total execution time, number of LLM calls, retries, and token usage are recorded from execution logs where available, allowing quality benefits to be weighed against computational and human-review costs.The execution performance measurements for the evaluated runs are reported in Table IX.

TABLE IX
EXECUTION PERFORMANCE

Run	Total Time (min)	LLM Calls	Retries	Tokens
Item Listing	36.4	24	4	31,800
Authentication	52.7	31	6	42,600
Shopping Cart	56.8	34	7	47,900
Baseline	31.2	18	3	25,400

VIII. DISCUSSION

The Item Listing case study shows the seven-agent workflow is feasible: all seven stages executed, produced reviewable

artifacts, and four required human-requested revisions that caught issues in requirements, architecture, UI/UX, and implementation before they propagated downstream.

This is a trade-off: human approval adds reviewer effort and execution time, since stages may be regenerated, but that effort buys the chance to catch problems early. The right balance depends on whether that reduction justifies the extra time.

Checkpointing has a similar trade-off: storage and orchestration overhead against avoiding a full restart after interruption. A real failure's recovery is evidence it works, though controlled trials must confirm reliability.

Security and QA results confirm executable verification works but shouldn't be overread: a zero-finding scan doesn't establish detection recall, and two passing tests don't establish broad correctness stronger claims need controlled vulnerabilities, broader tests, and coverage data.

The Domain Agent needs the same caution. Retrieval may improve requirement completeness by surfacing domain specific constraints, but that benefit has to be shown through RAG-ON versus RAG-OFF runs, not inferred just because retrieved information was present.

Finally, human approval's value should be tested, not assumed. If Human-Gates ON produces fewer downstream defects for an acceptable rise in reviewer effort and execution time, that supports the proposed approval architecture. But if some stages rarely need intervention, selectively approving only critical stages could be a more efficient alternative to approving every stage by default.

IX. CONCLUSION AND FUTURE WORK

This paper presented AutoForge, a seven-agent, human-in-the-loop pipeline for e-commerce web application development, orchestrated as a checkpointed LangGraph state graph. All seven agents were evaluated end-to-end across three e-commerce features with different complexity levels: Item Listing, Authentication/Authorization, and Shopping Cart. The evaluation covered artifact generation, human review, QA testing, security validation, checkpoint recovery, and domain retrieval. Every stage produced a versioned artifact with its own human decision, including revisions a reviewer requested at four of the seven stages, and the security scan and test run reported genuine, checkable results rather than a projected capability.

Future work follows three directions: expanding the evaluation to a larger and more diverse set of features toward benchmark-style reporting as in SWE-bench [9], generalizing beyond e-commerce by swapping the Domain Agent's knowledge base into other bounded domains (e.g., learning management, hospital management); and adding an integrated code editor to the review interface so reviewers can edit generated code directly at each gate.

REFERENCES

[1] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, *et al.*, "Chatdev: Communicative agents for software development," in *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pp. 15174–15186, 2024.

[2] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, S. Yau, Z. Lin, L. Zhou, *et al.*, "Metagpt: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations*, vol. 2024, pp. 23247–23275, 2024.

[3] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[4] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," *Advances in Neural Information Processing Systems*, vol. 37, pp. 50528–50652, 2024.

[5] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, *et al.*, "Openhands: An open platform for ai software developers as generalist agents," in *International Conference on Learning Representations*, vol. 2025, pp. 65882–65919, 2025.

[6] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better llm agents," *arXiv preprint arXiv:2402.01030*, 2024.

[7] H. N. Phan, T. N. Nguyen, P. X. Nguyen, and N. D. Bui, "Hyperagent: Generalist software engineering agents to solve coding tasks at scale," *arXiv preprint arXiv:2409.16299*, 2024.

[8] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying llm-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024.

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?," in *International Conference on Learning Representations*, vol. 2024, pp. 54107–54157, 2024.

[10] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "Agentcoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint arXiv:2312.13010*, 2023.

[11] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "Repocoder: Repository-level code completion through iterative retrieval and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2471–2484, 2023.

[12] J. He, C. Treude, and D. Lo, "Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–30, 2025.

[13] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 342–352, IEEE, 2025.

[14] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024.

[15] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA Journal of Automatica Sinica*, vol. 12, no. 2, pp. 317–334, 2025.

[16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.

[17] A. Hemmat, M. Sharbaf, S. Kolahdouz-Rahimi, K. Lano, and S. Y. Tehrani, "Research directions for using llm in software requirement engineering: A systematic review," *Frontiers in Computer Science*, vol. 7, p. 1519437, 2025.

[18] O. C. Gotel and C. Finkelstein, "An analysis of the requirements traceability problem," in *Proceedings of IEEE international conference on requirements engineering*, pp. 94–101, IEEE, 1994.

[19] J. Mucha, A. Kaufmann, and D. Riehle, "A systematic literature review of pre-requirements specification traceability," *Requirements Engineering*, vol. 29, no. 2, pp. 119–141, 2024.

[20] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[21] K. Pelluru, "Langchain & langgraph in production: Architectures for multi-agent llm systems," *Journal of Data and Digital Innovation (JDDI)*, vol. 2, no. 3, pp. 1–9, 2025.

[22] Chroma, "Chroma documentation." [Online]. Available: https://docs.trychroma.com/, 2026. Accessed: Aug. 25, 2026.